BENEMÉRITA UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
Departamento de Ciencias Computacionales

Actividad de Aprendizaje 14. Programa 7.
Paginación Simple

2026-B
Sección D09
Sistemas Operativos
Código de la materia: IL366

P R E S E N T A (N)
ICOM - Velázquez Vazquéz Valeria - 224785718

PROFESOR(A)
Becerra Velázquez Violeta del Rocío

Guadalajara, Jalisco, Mx.
26 de Octubre de 2026

Índice

Objetivo de la actividad..... 3
Notas acerca del lenguaje.....3
Arquitectura y estructuras de datos.....3
Reto principal y cómo se solucionó.....4
Capturas de pantalla.....5
Conclusiones.....5
Bibliografía..... 5

Tabla de Imágenes

Ilustración 1. Solicitud del número de procesos y del valor del Quantum.....5
Ilustración 2. Ventana principal con la memoria dividida en marcos.....5
Ilustración 3. Tabla de páginas desplegada con la tecla T.....5
Ilustración 4. Tabla de procesos (BCP) desplegada con la tecla B.....5
Ilustración 5. Tabla final de procesos al terminar la simulación.....5

Objetivo de la Actividad

Implementar la técnica de administración de memoria llamada paginación simple sobre el simulador con planificación Round-Robin de la Actividad 10. Hasta el programa anterior la memoria solo admitía un máximo de cinco procesos (entre ejecución, listos y bloqueados); ahora la capacidad la determina el número de marcos libres: un proceso entra a memoria únicamente si todas sus páginas caben en marcos disponibles.

El simulador trabaja con una memoria de 230 unidades dividida en 46 marcos de 5 unidades, de los cuales 5 (del 41 al 45) los ocupa el sistema operativo y 41 quedan para los procesos. Cada proceso recibe de forma aleatoria su tiempo máximo estimado, su operación y su tamaño (entre 6 y 26), y se divide en páginas de 5 unidades. Se conservan los requerimientos del programa 5, con la interrupción por entrada/salida ahora con un tiempo de bloqueo de 6, y se agrega la tecla "T", que pausa la simulación y muestra la tabla de páginas de cada proceso y los marcos libres.

Notas acerca del Lenguaje

Se continuó con Python y la biblioteca gráfica PySide6 (Qt). La novedad principal en la interfaz es el widget MemoriaWidget, una clase propia que hereda de QWidget y sobrescribe paintEvent para dibujar con QPainter los 46 marcos en dos columnas, numerados de 0 a 45, tal como en la figura del enunciado. Cada marco se rellena en proporción a las unidades que realmente usa, de modo que la fragmentación interna de la última página de cada proceso es visible: sin color si está libre, negro para el sistema operativo, azul si el proceso está listo, rojo si está en ejecución y morado si está bloqueado. Dentro de cada marco se escribe el identificador del proceso y las unidades ocupadas (por ejemplo, P3 4/5).

La tecla T abre una ventana de diálogo (QDialog) con la tabla de páginas, y de la misma forma que la tecla B, solo se cierra con la tecla "C", momento en que la simulación continúa donde se quedó. Los datos iniciales (número de procesos y valor del Quantum) se solicitan con QInputDialog.getInt.

Arquitectura y Estructuras de Datos

Se conservó el patrón Modelo-Vista-Controlador. La clase Process (@dataclass) incorporó el atributo size y la lista frames, que funciona como la tabla de páginas del proceso: la posición i de la lista contiene el número de marco donde está la página i. También se agregaron la propiedad num_pages, que calcula el número de páginas con una división techo (por ejemplo, un proceso de tamaño 22 requiere 5 páginas: cuatro de 5 unidades y una de 2), y el método page_usage(i), que devuelve las unidades ocupadas por la página i y permite dibujar el marco de forma proporcional.

La memoria se representa en SimulatorEngine con la lista frame_owner de 46 elementos: cada posición guarda None si el marco está libre, la cadena "SO" si pertenece al sistema operativo (marcos 41 a 45) o el identificador del proceso que lo ocupa. Sobre esta estructura se construyeron los métodos get_free_frames (marcos libres), _admit_new_processes (planificador a largo plazo),

_release_frames (liberación al terminar un proceso), get_frame_table (estado de cada marco para la pantalla) y get_page_tables (tablas de páginas para la tecla T). Las colas de nuevos, listos, bloqueados y terminados siguen siendo listas FIFO, y la cola de listos se maneja como carrusel con Round-Robin.

Página	Marco	Unidades usadas
0	0	5 / 5
1	1	5 / 5
2	7	3 / 5

Tabla 1. Ejemplo de tabla de páginas de un proceso de tamaño 13 (3 páginas, marcos no contiguos)

Reto Principal y Cómo se Solucionó

El principal reto fue sustituir el límite fijo de cinco procesos por la restricción de marcos libres. El planificador a largo plazo ahora revisa el primer proceso de la cola de nuevos: si hay marcos libres suficientes para todas sus páginas lo admite, le asigna los primeros marcos disponibles y lo pasa a listos; si falta aunque sea un marco, el proceso no entra y espera. Se decidió respetar el orden de llegada y no saltar al siguiente proceso, aunque este sí cupiera, para evitar que un proceso grande quede esperando indefinidamente. Esa es también la razón por la que la pantalla muestra siempre el ID y el tamaño del proceso que está por ingresar, junto con los marcos libres, para visualizar cuántos marcos hacen falta.

Otro punto delicado fue la liberación de marcos. Los procesos bloqueados conservan sus marcos mientras esperan su entrada/salida de 6 unidades, porque siguen en memoria; en cambio, al terminar un proceso (normalmente o por error con la tecla "E") se liberan sus marcos de inmediato y se intenta admitir procesos de la cola de nuevos en ese mismo instante. Como las páginas no tienen que ser contiguas, el algoritmo simplemente toma los marcos libres de menor número, de modo que un mismo proceso puede quedar repartido en marcos separados.

Para mostrar la memoria en todo momento se separó la lógica de la presentación: el motor devuelve por cada marco una tupla (estado, proceso, unidades usadas) y el widget solo se encarga de dibujarla. Las teclas B y T comparten el mismo método para pausar la simulación y reanudarla únicamente si no estaba pausada previamente con "P". Para validar el motor sin la interfaz se hicieron pruebas con procesos aleatorios, interrupciones, errores y procesos nuevos, comprobando en cada unidad de tiempo que el número de marcos ocupados coincidiera con el total de páginas de los procesos en memoria, que los marcos del sistema operativo no se modificaran y que al terminar los 41 marcos quedaran libres.

Capturas de Pantalla

Las siguientes ilustraciones corresponden a la ejecución del programa con paginación simple y las teclas I, E, P, C, N, B y T.

[Insertar aquí la captura de los cuadros de diálogo: Número de procesos y Valor del Quantum]

Ilustración 1. Solicitud del número de procesos y del valor del Quantum

[Insertar aquí la captura de la ventana principal con la memoria dividida en marcos]

Ilustración 2. Ventana principal con la memoria dividida en marcos

[Insertar aquí la captura de la tabla de páginas mostrada con la tecla T]

Ilustración 3. Tabla de páginas desplegada con la tecla T

[Insertar aquí la captura de la tabla BCP mostrada con la tecla B]

Ilustración 4. Tabla de procesos (BCP) desplegada con la tecla B

[Insertar aquí la captura de la tabla final al terminar la simulación]

Ilustración 5. Tabla final de procesos al terminar la simulación

Conclusiones

La paginación simple cambia la forma de decidir cuántos procesos caben en memoria: ya no importa el número de procesos, sino si existen marcos suficientes para todas sus páginas. Al trabajar con el simulador se observa que no hay fragmentación externa, porque cualquier marco libre sirve para cualquier página, y que la única pérdida de espacio es la fragmentación interna de la última página de cada proceso, que se aprecia claramente en los marcos rellenos solo parcialmente.

También se confirmó que la separación entre el motor de simulación y la interfaz facilitó el cambio: la administración de marcos quedó en SimulatorEngine y Process, y la ventana solo dibuja el estado que el motor le entrega. Una limitación de la técnica, visible en el programa, es que un proceso debe esperar en la cola de nuevos hasta que se libere la cantidad exacta de marcos que necesita, aunque exista memoria libre dispersa de otros tamaños, lo que motiva el estudio posterior de la memoria virtual.

Bibliografía

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10a ed.). Wiley.

Stallings, W. (2018). Operating Systems: Internals and Design Principles (9a ed.). Pearson.

The Qt Company. (2024). PySide6 Reference Documentation — QPainter, QWidget, QDialog y QInputDialog. Recuperado de la documentación oficial de Qt for Python.